

C03 – AT1

MLA 0302 - REINFORCEMENT LEARNING

1. Reinforcement Learning Framework – Agent, Environment, State, Action and Reward

Answer

Reinforcement Learning (RL) is a learning method where an **agent learns by interacting with an environment**.

The main components are:

1. **Agent** – The learner or decision-maker.
2. **Environment** – The world in which the agent operates.
3. **State (S)** – Current situation of the agent.
4. **Action (A)** – Decision taken by the agent.
5. **Reward (R)** – Feedback received after taking an action.
6. **Policy** – Strategy used by the agent to select actions.

The interaction is:

State → Action → Reward → New State

The agent tries to maximize the **cumulative reward**, which is the total reward collected over time.

A higher cumulative reward means better long-term decision-making.

Example

A robot moves in a grid:

- State = robot's current position

- Action = Up, Down, Left, Right
- Reward = +10 for reaching the goal
- Reward = -1 for hitting an obstacle

Simple Code

```
import random
```

```
state = 0
```

```
total_reward = 0
```

```
for step in range(5):
```

```
    action = random.choice(["Left", "Right"])
```

```
    if action == "Right":
```

```
        reward = 1
```

```
        state += 1
```

```
    else:
```

```
        reward = -1
```

```
    total_reward += reward
```

```
    print("State:", state, "Action:", action, "Reward:", reward)
```

```
print("Cumulative Reward:", total_reward)
```

Sample Output

```
State: 1 Action: Right Reward: 1
```

```
State: 0 Action: Left Reward: -1
```

```
State: 1 Action: Right Reward: 1
```

State: 2 Action: Right Reward: 1

State: 3 Action: Right Reward: 1

Cumulative Reward: 3

2. Decision-Making as a Markov Decision Process (MDP)

Answer

An **MDP** is a mathematical framework used to model decision-making problems in RL.

An MDP consists of:

1. State Space (S)

Set of all possible situations.

Example:

$S = \{\text{Position 1, Position 2, Position 3}\}$

2. Action Space (A)

Set of actions available to the agent.

$A = \{\text{Left, Right}\}$

3. Transition Probability P

It represents the probability of moving from one state to another after taking an action.

4. Reward Function R

It defines the reward obtained after an action.

5. Discount Factor γ

It determines the importance of future rewards.

A complete MDP can be represented as:

Simple Code

```
states = ["A", "B", "C"]
```

```
actions = ["Left", "Right"]
```

```
reward = {  
    ("A", "Right"): 1,  
    ("B", "Right"): 5,  
    ("C", "Left"): -1  
}
```

```
print("States:", states)  
print("Actions:", actions)  
print("Reward A->Right:", reward[("A", "Right")])  
print("Discount Factor:", 0.9)
```

Output

States: ['A', 'B', 'C']

Actions: ['Left', 'Right']

Reward A->Right: 1

Discount Factor: 0.9

3. Policy and Value Functions in RL

Answer

A **policy** defines how an agent selects an action from a state.

There are two important value functions:

State-Value Function

It estimates the expected future reward from a state.

Action-Value Function

It estimates the expected future reward for taking action a in state s .

The **Bellman Equation** breaks a long-term problem into immediate reward plus future value.

For state value:

For Q-value:

Simple Code

```
reward = 10
```

```
gamma = 0.9
```

```
future_value = 5
```

```
value = reward + gamma * future_value
```

```
print("State Value:", value)
```

Output

State Value: 14.5

This means the current value is the **immediate reward 10 + discounted future value 4.5**.

4. Exploration–Exploitation Trade-off

Answer

RL agents must decide between:

- **Exploration** – Trying new actions to discover better rewards.
- **Exploitation** – Choosing the action currently known to give the highest reward.

This is called the **exploration–exploitation trade-off**.

ϵ -Greedy

With ϵ -greedy:

- Probability $\epsilon \rightarrow$ explore

- Probability $1-\epsilon \rightarrow$ exploit

For example:

$\epsilon = 0.1$

10% exploration

90% exploitation

Softmax

Softmax selects actions according to their Q-values.

Here T is the temperature.

Method	Advantage	Limitation
ϵ -greedy	Simple	Exploration is random
Softmax	Uses Q-values	More computationally complex

Simple Code

```
import random
```

```
q_values = [2, 5, 3]
```

```
epsilon = 0.1
```

```
if random.random() < epsilon:
```

```
    action = random.randint(0, 2)
```

```
    print("Exploration")
```

```
else:
```

```
    action = q_values.index(max(q_values))
```

```
    print("Exploitation")
```

```
print("Selected Action:", action)
```

Sample Output

Exploitation

Selected Action: 1

5. Reward Shaping in Reinforcement Learning

Answer

Reward shaping means adding useful intermediate rewards to help the agent learn faster.

Without reward shaping, an agent may receive a reward only after completing the entire task.

For example, in a robot navigation problem:

Reaching goal = +10

Moving closer to goal = +1

Hitting obstacle = -5

Reward shaping helps the agent understand which actions are useful.

Benefits

1. Speeds up learning.
2. Provides feedback during intermediate steps.
3. Reduces unnecessary exploration.
4. Helps avoid poor policies.
5. Improves convergence.

Example Code

```
distance = 5
```

```
for step in range(5):
```

```
    distance -= 1
```

```
if distance == 0:
    reward = 10
else:
    reward = 1

print("Distance:", distance, "Reward:", reward)
```

Output

Distance: 4 Reward: 1

Distance: 3 Reward: 1

Distance: 2 Reward: 1

Distance: 1 Reward: 1

Distance: 0 Reward: 10

The intermediate rewards guide the agent toward the goal.

6. Challenges of RL in Real-World Robotics

Answer

Applying RL to real robots is difficult because robots operate in unpredictable environments.

Major challenges

1. Safety

Wrong actions can damage the robot or surrounding objects.

Example:

Robot moves too fast → Collision

Therefore, safety restrictions are required.

2. Sample Efficiency

RL may require thousands or millions of interactions.

A real robot cannot repeatedly perform dangerous experiments.

3. Environment Variability

Real environments can change because of:

- Lighting
- Objects
- People
- Weather
- Sensor noise

4. Hardware Limitations

Robots have limited:

- Battery
- Processing power
- Sensors

5. Sim-to-Real Gap

A policy trained in simulation may not work perfectly on a real robot.

Simple Code

```
import random
```

```
for step in range(5):
```

```
    distance = random.randint(1, 10)
```

```
    if distance < 3:
```

```
        print("STOP - Safety Risk")
```

```
    else:
```

```
        print("Move")
```

Sample Output

Move

Move

STOP - Safety Risk

Move

STOP - Safety Risk

Safety mechanisms prevent dangerous actions.

7. Influence of Discount Factor γ

Answer

The **discount factor γ (gamma)** determines how much the agent values future rewards.

Low γ

Example:

The agent focuses mainly on immediate rewards.

High γ

Example:

The agent considers future rewards more strongly.

Gamma	Decision-making
0	Only immediate reward
0.3	Mostly short-term
0.7	Balanced
0.9	Long-term
1.0	Future rewards highly important

Example

Suppose:

Immediate reward = 5

Future reward = 10

With $\gamma = 0.5$:

With $\gamma = 0.9$:

Code

```
reward = 5
```

```
future_reward = 10
```

```
for gamma in [0.5, 0.9]:
```

```
    value = reward + gamma * future_reward
```

```
    print("Gamma:", gamma, "Value:", value)
```

Output

```
Gamma: 0.5 Value: 10.0
```

```
Gamma: 0.9 Value: 14.0
```

In episodic tasks, γ controls how much future rewards within the episode influence current decisions.

8. Model-Free RL vs Model-Based RL

Answer

RL methods can be divided into **model-free** and **model-based** approaches.

Model-Free RL

The agent directly learns which actions are good without learning the environment model.

Examples:

- Q-learning

- SARSA
- Policy Gradient

Advantages

- Simple implementation.
- Works well in complex environments.
- Does not require transition probabilities.

Limitations

- Requires many samples.
- Can be slow to learn.

Model-Based RL

The agent learns or uses a model of the environment.

It learns:

Current State + Action → Next State + Reward

Advantages

- More sample efficient.
- Can plan future actions.
- Useful when interactions are expensive.

Limitations

- Model can be inaccurate.
- More computationally complex.

Feature	Model-Free	Model-Based
Environment model	Not required	Required
Learning	Usually slower	Usually faster
Planning	Limited	Strong

Feature	Model-Free	Model-Based
Complexity	Lower	Higher
Sample efficiency	Lower	Higher

Simple Code

```

model = {
    "A": {"Right": "B"},
    "B": {"Right": "C"}
}

state = "A"
action = "Right"

next_state = model[state][action]

print("Current State:", state)
print("Action:", action)
print("Next State:", next_state)

```

Output

```

Current State: A
Action: Right
Next State: B

```

9. Q-Learning Update and Convergence

Answer

Q-learning is an off-policy, model-free RL algorithm.

It learns the best action-value function $Q(s,a)$.

The update equation is:

Where:

- α = learning rate
- γ = discount factor
- R = reward
- s' = next state

Example

Suppose:

Old $Q = 0$

Reward = 10

Gamma = 0.9

Best next $Q = 5$

Learning rate = 0.5

Then:

Python Code

```
q = 0
```

```
alpha = 0.5
```

```
gamma = 0.9
```

```
reward = 10
```

```
next_q = 5
```

```
for i in range(5):
```

```
q = q + alpha * (reward + gamma * next_q - q)
print("Iteration", i + 1, "Q-value:", round(q, 2))
```

Output

Iteration 1 Q-value: 7.25

Iteration 2 Q-value: 10.88

Iteration 3 Q-value: 12.69

Iteration 4 Q-value: 13.59

Iteration 5 Q-value: 14.05

The Q-value gradually approaches the target value:

This gradual stabilization demonstrates **convergence**.

10. On-Policy vs Off-Policy Learning

Answer

RL algorithms can be classified into **on-policy** and **off-policy** methods.

On-Policy Learning

An on-policy algorithm learns about the same policy that it follows.

Example:

SARSA

The agent learns:

using the action actually selected by its current policy.

Advantages

- Learns from the current behavior.
- Useful when the actual behavior policy is important.
- Can learn safer behavior.

Limitations

- Learning can be slower.
- Exploration affects the learned policy.

Off-Policy Learning

An off-policy algorithm learns the optimal policy while potentially following another behavior policy.

Example:

Q-learning

It uses:

to estimate the best next action.

Advantages

- Can learn the optimal policy while exploring.
- Can reuse previous experiences.
- Often more flexible.

Limitations

- Can be unstable with function approximation.
- May select risky actions during exploration.

Comparison

Feature	On-Policy	Off-Policy
Example	SARSA	Q-learning
Learns	Current policy	Different/optimal policy
Exploration	Part of learned behavior	Can be separate
Flexibility	Lower	Higher
Experience reuse	Limited	Better

Feature	On-Policy	Off-Policy
Safety	Often safer	Can be more aggressive

Simple Code

```
q_sarsa = 2
```

```
q_learning = 2
```

```
reward = 5
```

```
gamma = 0.9
```

```
next_q = 4
```

```
# SARSA
```

```
q_sarsa = q_sarsa + 0.5 * (reward + gamma * next_q - q_sarsa)
```

```
# Q-learning
```

```
best_next_q = 6
```

```
q_learning = q_learning + 0.5 * (reward + gamma * best_next_q - q_learning)
```

```
print("SARSA Q-value:", q_sarsa)
```

```
print("Q-learning Q-value:", q_learning)
```

Output

```
SARSA Q-value: 3.3
```

```
Q-learning Q-value: 3.2
```